

Digital Clinic Management System

Advanced Program

Stored Procedure:

SQL

```
CREATE PROCEDURE CreateInvoice(IN appointment_id INT, IN consultationFees
DECIMAL(10,2))
BEGIN
    DECLARE totalAmount DECIMAL(10,2) DEFAULT 0.00;
    DECLARE currentProcedureAmount DECIMAL(10,2);

    DECLARE done INT DEFAULT 0;
    DECLARE cur CURSOR FOR (
        SELECT Amount AS OrderItemAmounts
        FROM ConsultationProcedures NATURAL JOIN Procedures NATURAL JOIN
Consultations
        WHERE AppointmentID = appointment_id
    );
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    SET totalAmount = totalAmount + consultationFees;

    OPEN cur;

    REPEAT
        FETCH cur INTO currentProcedureAmount;
        IF done = 0 THEN
            SET totalAmount = totalAmount + currentProcedureAmount;
        END IF;
    UNTIL done END REPEAT;

    CLOSE cur;
```

```
INSERT INTO Invoices(Amount, AppointmentID) VALUES (totalAmount,
appointment_id);
END;
```

Trigger:

```
SQL
CREATE TRIGGER MarkAppointmentComplete
  BEFORE UPDATE ON Appointments
  FOR EACH ROW
BEGIN
  IF NEW.AppointmentStatus = "COM" AND OLD.AppointmentStatus <> "COM" THEN
    SET @consultationFees = (SELECT ConsultationFees AS OrderItemAmounts FROM
Doctors WHERE DoctorID = NEW.DoctorID);
    CALL CreateInvoice(NEW.AppointmentID, @consultationFees);
  END IF;
END;
```

Transactions:

Doctor signup + Clinic creation by admin doctor:

```
TypeScript
const conn = await getDbConnection();

if (!conn) return res.status(500).end();

await conn.query(
  "SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;"
);

await conn.beginTransaction();

try {
  ... //Other code
  const createClinicQuery =
    "INSERT INTO Clinics(ClinicName, Zipcode) VALUES (?, ?)";
  const createClinicValues = [clinicName, zipcode];
```

```

    const [{ insertId: newClinicId = 0 } = {}] =
      await conn.execute<ResultSetHeader>(
        createClinicQuery,
        createClinicValues
      );
    _clinicId = newClinicId;
  } else {
    if (!clinicId) {
      return res.status(400).json({ error: "ClinicID is required" });
    }
    _clinicId = clinicId;
  }
}

const createDoctorQuery =
  "INSERT INTO Doctors(FirstName, LastName, EmailAddress, Specialization, ConsultationFees, Role, ClinicID) VALUES (?, ?, ?, ?, ?, ?, ?)";
const createDoctorValues = [
  firstName,
  lastName,
  emailAddress,
  specialization,
  consultationFees,
  role,
  _clinicId
];

const [{ insertId: newDoctorId = 0 } = {}] =
  await conn.execute<ResultSetHeader>(
    createDoctorQuery,
    createDoctorValues
  );

const query = "SELECT *FROM Doctors WHERE doctorId = ?";
const values = [newDoctorId];

const [rows] = await conn.execute<Doctor[]>(query, values);

await conn.commit();

return res.status(201).json({
  data: rows[0] || {}
});
} catch (error) {
  console.error("Error during signup:", error);
}

```

```

    await conn.rollback();
    return res.status(500).json({ error: "Internal server error" });
  } finally {
    conn.release();
  }
}

```

Doctor profile update + Clinic profile update by admin doctor:

TypeScript

```

const conn = await getDbConnection();

if (!conn) return res.status(500).end();

await conn.query(
  "SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;"
);

await conn.beginTransaction();

try {
  const doctorUpdateQuery =
    "UPDATE Doctors SET firstName = ?, lastName = ?, specialization = ?,
consultationFees = ? WHERE doctorId = ?;";
  const doctorUpdateValues = [
    firstName,
    lastName,
    specialization,
    consultationFees,
    doctorId
  ];

  const clinicUpdateQuery =
    "UPDATE Clinics SET clinicName = ?, zipcode = ? WHERE clinicId = ?;";
  const clinicUpdateValues = [clinicName, zipcode, clinicId];

  await Promise.all([
    conn.execute<Clinic[]>(clinicUpdateQuery, clinicUpdateValues),
    conn.execute<Doctor[]>(doctorUpdateQuery, doctorUpdateValues)
  ]);

  const doctorQuery =

```

```

        "SELECT firstName, lastName, role, emailAddress, specialization,
consultationFees FROM Doctors WHERE doctorId = ?;";
        const doctorValues = [doctorId];

        const clinicQuery = "SELECT clinicName FROM Clinics WHERE clinicId = ?;";
        const clinicValues = [clinicId];

        const [[doctorRows], [clinicRows]] = await Promise.all([
            conn.execute<Clinic[]>(clinicQuery, clinicValues),
            conn.execute<Doctor[]>(doctorQuery, doctorValues)
        ]);

        await conn.commit();

        return res.status(200).json({
            data: { ...(doctorRows?.[0] || {}), ...(clinicRows?.[0] || {}) }
        });
    } catch (e) {
        await conn.rollback();
        console.error("Error updating doctor info:", e);
        return res
            .status(500)
            .json({ error: "Failed to update doctor information" });
    } finally {
        conn.release();
    }
}

```

Creation of consultation + Associated procedures:

TypeScript

```

const conn = await getDbConnection();
if (!conn) {
    return res
        .status(500)
        .json({ error: "Database connection failed" } as ConsultationResponse);
}

await conn.query("SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;");

await conn.beginTransaction();

try {
    const [result]: any = await conn.execute(

```

```

`INSERT INTO Consultations (
    AppointmentID,
    HeartRate,
    RespiratoryRate,
    Temperature,
    BloodOxygen,
    SystolicBP,
    DiastolicBP,
    Weight,
    Height,
    ChiefComplaints,
    Diagnosis
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);`,
[
    appointmentID,
    heartRate,
    respiratoryRate,
    temperature,
    bloodOxygen,
    systolicBP,
    diastolicBP,
    weight,
    height,
    chiefComplaints,
    diagnosis
]
);

const insertId = result.insertId as number;

if (Array.isArray(procedures) && procedures.length > 0) {
    const valuesPlaceholders = procedures.map(() => "(?, ?)").join(", ");
    const valuesParams: any[] = [];

    (procedures as number[]).forEach((pid) => {
        valuesParams.push(insertId, pid);
    });

    await conn.execute(
        `
        INSERT INTO ConsultationProcedures (ConsultationID, ProcedureID)
        VALUES ${valuesPlaceholders};
        `,
        valuesParams
    );
}

```

```

    );
}

const [rows] = await conn.execute<Consultation[]>(
    "SELECT * FROM Consultations WHERE ConsultationID = ?;",
    [insertId]
);

await conn.commit();

return res.status(201).json({ data: rows[0] } as ConsultationResponse);
} catch (err) {
    console.error(err);
    await conn.rollback();
    return res
        .status(500)
        .json({ error: "Internal server error" } as ConsultationResponse);
} finally {
    conn.release();
}

```

Updation of consultation + Associated procedures:

TypeScript

```

const conn = await getDbConnection();
if (!conn) {
    return res
        .status(500)
        .json({ error: "Database connection failed" } as ConsultationResponse);
}

await conn.query("SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;");

await conn.beginTransaction();

try {
    const [apptRows]: any = await conn.execute(
        "SELECT AppointmentStatus, DoctorID FROM Appointments WHERE AppointmentID = ?;",
        [appointmentID]
    );
    ... // Other code
}

```

```

const [result]: any = await conn.execute(
  `UPDATE Consultations SET
    AppointmentID = ?,
    HeartRate = ?,
    RespiratoryRate = ?,
    Temperature = ?,
    BloodOxygen = ?,
    SystolicBP = ?,
    DiastolicBP = ?,
    Weight = ?,
    Height = ?,
    ChiefComplaints = ?,
    Diagnosis = ?
  WHERE AppointmentID = ?;`,
  [
    appointmentID,
    heartRate,
    respiratoryRate,
    temperature,
    bloodOxygen,
    systolicBP,
    diastolicBP,
    weight,
    height,
    chiefComplaints,
    diagnosis,
    appointmentID
  ]
);

if (result.affectedRows === 0) {
  return res
    .status(404)
    .json({ error: "Consultation not found" } as ConsultationResponse);
}

const [consRows] = await conn.execute<Consultation[]>(
  "SELECT ConsultationID FROM Consultations WHERE AppointmentID = ?;",
  [appointmentID]
);

const consultationId = (consRows[0] || {}).ConsultationID as number;

if (Array.isArray(procedures)) {

```

```

const [existingRows] = await conn.execute<any[]>(
  "SELECT ProcedureID FROM ConsultationProcedures WHERE ConsultationID =
?;",
  [consultationId]
);

const existingIds = new Set(existingRows.map((r) => r.ProcedureID));
const newIds = new Set<number>(procedures);

const toDelete: number[] = Array.from(existingIds.difference(newIds));
const toInsert: number[] = Array.from(newIds.difference(existingIds));

if (toDelete.length > 0) {
  const placeholders = toDelete.map(() => "?").join(", ");
  await conn.execute(
    `
      DELETE FROM ConsultationProcedures
      WHERE ConsultationID = ?
      AND ProcedureID IN (${placeholders});
    `,
    [consultationId, ...toDelete]
  );
}
if (toInsert.length > 0) {
  const valuesPlaceholders = toInsert.map(() => "(?, ?)").join(", ");
  const valuesParams: any[] = [];
  toInsert.forEach((pid) => {
    valuesParams.push(consultationId, pid);
  });

  await conn.execute(
    `
      INSERT INTO ConsultationProcedures (ConsultationID, ProcedureID)
      VALUES ${valuesPlaceholders};
    `,
    valuesParams
  );
}

const [rows] = await conn.execute<Consultation[]>(
  "SELECT * FROM Consultations WHERE AppointmentID = ?;",
  [appointmentID]
);

```

```
    await conn.commit();

    return res.status(200).json({
      data: rows[0]
    } as ConsultationResponse);
  } catch (err) {
    console.error(err);
    await conn.rollback();
    return res
      .status(500)
      .json({ error: "Internal server error" } as ConsultationResponse);
  } finally {
    conn.release();
  }
}
```